

TACHYON

From Platform SDK to a Bounded Adaptive Harness

Enterprise architecture, ownership, governance, runtime design and phased implementation proposal

Core recommendation

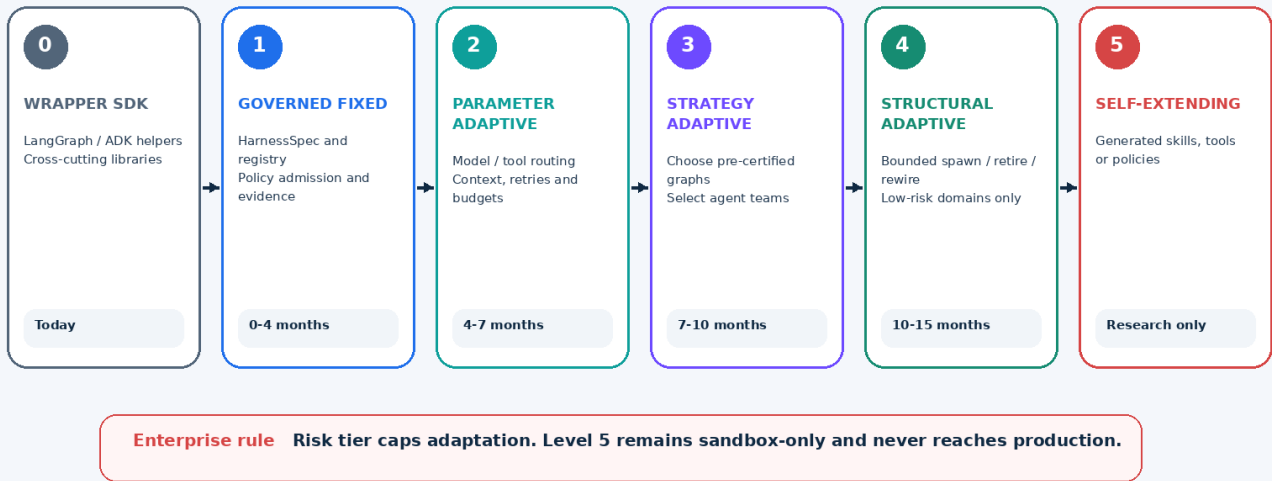
Do not implement unconstrained self-organizing agents. Evolve Tachyon into a bounded adaptive harness: an immutable enterprise safety kernel around a configurable domain harness, with runtime adaptation limited to a certified policy envelope.

Governed Agent = Model(s) + Domain Harness + Enterprise Safety Kernel + Runtime State

Adaptive Agent System = Governed Agent + Bounded Feedback Loop + Continuous Evidence

Adaptive Harness Maturity Roadmap

Move from framework wrappers to evidence-driven, policy-bounded adaptation.



Tachyon Adaptive Harness Proposal | July 2026

Prepared for Technology Leadership

July 2026 | Internal working proposal

Contents

Executive recommendation

1. Review of the adaptive engineering thesis
2. Definition and strategic position of the Tachyon harness
3. Current state and target capability model
4. Reference architecture
5. Design-time governance
6. Runtime governance and adaptive control loops
7. Platform, product-team and control ownership
8. Memory architecture: where Redis and MongoDB fit
9. Risk-tiered adaptation model
10. SDK and control-plane product proposal
11. Evidence, security and operational governance
12. Delivery roadmap and operating model
13. Measures of success
14. Risks, objections and responses
15. Leadership decisions requested

Appendices: HarnessSpec, use-case examples, presentation narrative, sources

Executive Recommendation

One-sentence thesis

The platform should own the invariant execution and control mechanisms; product teams should own domain intent and outcomes; runtime adaptation should be allowed only inside a versioned, observable and pre-certified envelope.

The video makes a valuable strategic point: fixed, pre-runtime agent graphs become brittle when the problem space, tools, available models and operating environment keep changing. It argues that the harness should become an evolving runtime output rather than a static input. That insight is directionally right for complex domains, but it is incomplete for a regulated enterprise because it does not solve auditability, privilege control, runtime certification or accountability. The proposal therefore adopts the insight but changes the implementation model: bounded adaptation rather than unconstrained emergence. [1][2]

Tachyon is already well positioned. The current SDK provides service clients, resilience patterns, LangGraph and Google ADK wrappers, A2A support, model routing, MCP integration and observability. These are the correct ingredients, but they are presently exposed mainly as libraries. The next strategic step is to turn them into a framework-neutral platform contract, governance control plane and runtime evidence system.

Recommended decisions

- **Adopt a platform definition of harness.** A harness is the policy-governed execution envelope that binds models to identity, context, memory, tools, orchestration, evaluation, resilience and evidence.
- **Build a canonical HarnessSpec.** Every agent or multi-agent workflow is declared, risk-tiered, validated, signed, registered and instantiated from this contract.
- **Separate immutable and adaptive concerns.** Identity, authorization, data boundaries, tool registration, audit, budget ceilings and kill controls never self-modify. Model routing, context shaping, retries and topology selection may adapt inside explicit limits.
- **Create three feedback loops.** A fast execution loop, a near-real-time safety loop and a slower learning/promotion loop. Runtime learning proposes changes; it does not silently promote them.
- **Own memory as a service contract.** The platform owns isolation, lineage, retention and pluggable adapters. Teams own what is remembered and the domain quality of retrieval.
- **Progress through adaptation levels.** Start with governed fixed harnesses, then parameter adaptation, then selection among pre-certified strategies. Dynamic structural adaptation is limited to low-risk use cases. Self-extension remains sandbox-only.

What success looks like

- Every run can be reconstructed from a signed harness version, model and tool versions, policy decisions, memory operations and adaptation receipts.
- Development teams stop rebuilding authentication, checkpointing, tracing, evaluator plumbing, model routing and tool-control logic.
- The platform can switch framework adapters or model providers without changing the governance contract.

- Adaptation improves task success, cost or latency while preserving control thresholds and rollback capability.
- Technology leadership can answer who authorized an action, why the topology changed, what evidence supported it and who owns the outcome.

1. Review of the Adaptive Engineering Thesis

1.1 What the video argues

Rajiv Chandegra contrasts fixed harness engineering with adaptive engineering. A fixed harness determines roles, tools, topology and handoffs before runtime. It is reliable, predictable and auditable, but suppresses variance. The adaptive model treats the real world as a complex system: agents interact locally, specialize, form clusters and create conventions; the engineer designs constraints and selection pressures rather than prescribing every interaction. The harness emerges, stabilizes, changes and may dissolve during runtime. [1][2]

The talk also identifies the central tradeoff. Fixed structures are appropriate for stable, well-defined problems; adaptive structures may be useful for moving problems that require probe-sense-respond behavior. It warns about attractor traps, agent monoculture, loss of legibility and the inability to predict runtime behavior in advance. [1]

Video proposition	Assessment	Enterprise implication
Fixed harnesses become brittle in moving environments.	Strong and directly relevant. Models, tools, policies and task conditions change faster than application release cycles.	Make selected harness parameters dynamically resolvable at runtime.
The harness should emerge from agent interaction.	Useful as a research direction, but too broad for material enterprise actions.	Permit bounded strategy selection, not unrestricted self-organization.
Engineer designs constraints rather than structure.	Correct as a shift in emphasis, not a complete replacement.	Platform defines invariants; domain teams define objectives and allowed degrees of freedom.
Horizontal intelligence can be higher leverage than model strength.	Conditionally true. Multi-agent gains depend on task structure; coordination can also degrade sequential tasks.	Classify the task before selecting single-agent, centralized, decentralized or hybrid topology. [15]
Adaptation reduces predictability and legibility.	This is the key adoption barrier in banking.	Replace one-time certification with runtime evidence, attestation, policy enforcement and replay.

1.2 Our position: bounded emergence

Strategic interpretation

We should not choose between a static factory and an uncontrolled flock. The enterprise pattern is bounded emergence: invariant safety boundaries plus adaptive execution choices, with every choice observable and attributable.

- **Use deterministic orchestration where the problem is complicated.** Regulated calculations, entitlement checks, transaction sequencing and mandatory approvals should remain explicit.

- **Use adaptive selection where the problem is complex.** Research, investigation, incident diagnosis, retrieval planning and workload decomposition can choose among safe strategies.
- **Treat adaptation as a controlled act.** A topology change is equivalent to a runtime configuration change and must have policy authorization, provenance and rollback.
- **Do not equate more agents with more intelligence.** Google Research found that multi-agent systems help parallelizable work but can degrade sequential reasoning; topology must be task-aware. [15]

2. Definition and Strategic Position of the Tachyon Harness

2.1 Proposed enterprise definition

Enterprise Agent Harness

A policy-governed, observable and durable execution envelope that binds one or more models to domain goals, identities, tools, context and memory, orchestration, safety controls, evaluation, resilience and operational feedback.

This definition is intentionally broader than an SDK wrapper. LangGraph describes itself as an orchestration runtime with durable execution, streaming, human-in-the-loop and persistence; ADK exposes runtime, session, memory, callbacks, plugins and evaluation capabilities. Tachyon should not duplicate every native feature. It should create a canonical enterprise contract across these frameworks and enforce the controls that individual frameworks cannot own for the enterprise. [3][4][5][6]

2.2 Better formula than “harness + model = agent”

- **Agent definition.** Model(s) + domain harness + enterprise safety kernel + runtime state.
- **Agent identity.** A registered principal with a capability profile, owner, purpose, risk tier and delegated authority chain - not merely a system prompt.
- **Multi-agent system.** A set of agent identities and interaction contracts coordinated through a topology and shared evidence model.
- **Adaptive harness.** A harness whose permitted runtime parameters or topology may change based on observed conditions, under a certified policy envelope.
- **Business/product domain.** The bounded context that owns outcome semantics, data, tools, evaluators, runbooks and residual risk.

2.3 Design principles

Principle	Meaning
Invariants over implementations	Govern identity, authority, evidence and outcomes without forcing every team into one framework.
Framework-neutral contract	Compile the HarnessSpec into LangGraph, ADK or future runtimes through adapters.
Least agency	Grant the minimum models, tools, data, memory and

Principle	Meaning
	adaptation needed for the use case.
Evidence by default	A run that cannot produce an evidence receipt is not production-ready.
Runtime controls are first-class	Pause, resume, replay, rewind, fallback and kill are platform capabilities, not application afterthoughts.
Learning is separated from promotion	Systems may learn or propose, but controlled promotion changes production behavior.
Domain accountability remains local	The platform cannot own whether an underwriting, fraud, service or operations outcome is correct.
Portability over database coupling	Memory and state are semantic interfaces, not a single Redis or MongoDB choice.

3. Current State and Target Capability Model

3.1 Current Tachyon SDK foundation

Based on the current SDK overview, Tachyon already provides a high-performance asynchronous Python client for platform services, enterprise resilience patterns, a LangGraph/Google ADK A2A toolkit, model routing, MCP support and OpenTelemetry-based observability. This is a strong engineering base. The strategic gap is not another agent framework; it is the missing control-plane and governance product around these capabilities.

Current capability	Target role in adaptive harness	Required evolution
Service clients	Canonical resource and tool clients	Add identity propagation, policy context, telemetry and declared data classification.
Resilience4P	Execution reliability layer	Make retry, breaker, rate and bulkhead policies runtime-resolvable within limits.
A2A framework	Inter-agent interoperability plane	Add agent registry, capability cards, trust policy, delegation and evidence correlation. A2A standardizes agent-to-agent communication; MCP remains agent-to-tool. [7][8]
Model Router	Adaptive model decision point	Add outcome-aware routing, approved model sets, budgets, shadow evaluation and decision receipts.
MCP Core	Governed tool plane	Add server registration, short-lived identity, scope enforcement, schema validation, tool risk tier and kill controls.
Observability	Continuous evidence plane	Adopt canonical events for agent, model, tool, memory, policy, evaluator and adaptation operations. OTEL provides vendor-neutral GenAI telemetry foundations. [9]
Framework wrappers	Runtime adapters	Retain native features while compiling from a common HarnessSpec.

3.2 Capabilities to add

- HarnessSpec schema, compiler, validator, registry and signed artifact.
- Agent identity and capability registry with owner, purpose, risk tier and delegable scopes.
- Policy admission at design time and policy decision points at runtime.
- Memory Service API and portable state/checkpoint contracts.
- Evaluation and fitness service for offline, shadow, online and post-run scoring.
- Adaptation controller with allowed actions, evidence, canarying and rollback.
- Runtime evidence receipt and replay service.
- Governance console for approvals, exceptions, incidents and residual-risk sign-off.
- Canonical runtime event model across LangGraph, ADK, A2A, MCP and model providers.

4. Reference Architecture

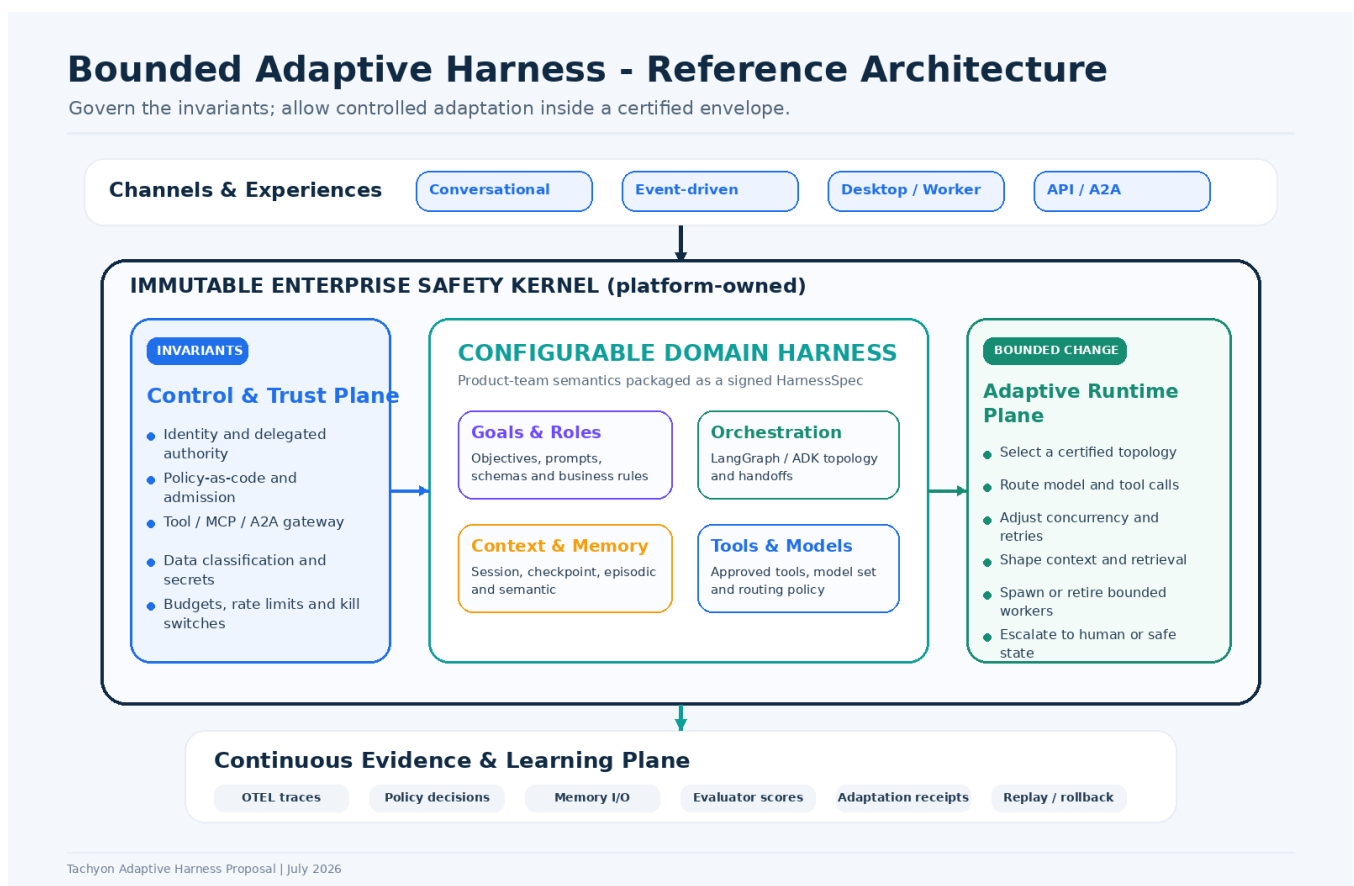


Figure 1. The immutable safety kernel surrounds a configurable domain harness and a bounded adaptive runtime plane.

4.1 Immutable enterprise safety kernel - platform owned

- Agent and user identity, delegation chain and workload identity.
- Authorization, policy-as-code, data classification and purpose limitation.
- Approved model, tool, MCP server and A2A peer registries.
- Secrets handling, network controls, isolation, budgets and rate ceilings.
- Trace and evidence requirements, PII redaction and retention controls.
- Pause, kill, quarantine, fallback and safe-state mechanisms.
- No agent or adaptation controller may modify this kernel at runtime.

4.2 Configurable domain harness - product team owned

- Business objective, completion contract and failure semantics.
- Prompts, roles, schemas, graph or agent-team definition and handoffs.
- Domain tools, data entitlements and systems-of-record integration.
- Context assembly, retrieval strategy, memory write policy and retention need.
- Golden datasets, business evaluators, quality thresholds and residual risk.
- Runbooks, user experience, human approvals and incident ownership.

4.3 Adaptive runtime plane - jointly governed

The adaptive plane does not have arbitrary authority. It chooses from explicit degrees of freedom. Examples of allowed actions include choosing a pre-certified graph, selecting an approved model, changing retrieval depth, adjusting retry strategy, changing concurrency, spawning a bounded worker or escalating to a human. Examples of prohibited actions include adding an unregistered tool, expanding data access, weakening policy, exceeding a budget ceiling, changing an evaluator threshold or writing to an unapproved memory namespace.

4.4 Continuous evidence and learning plane

Every material runtime choice produces an event and a signed adaptation receipt containing the previous configuration, triggering signals, policy decision, selected action, expected benefit, actual result and rollback pointer. This converts runtime dynamism from an invisible behavior into an auditable control-plane operation.

5. Design-Time Governance

5.1 Design-time lifecycle

1. Classify the use case: business domain, data classes, user population, materiality, reversibility and regulatory obligations.
2. Assign risk tier and maximum adaptation level.
3. Author the HarnessSpec, domain eval pack, tool contracts, memory policy and operational ownership.
4. Compile the spec into a framework-specific artifact while injecting platform callbacks, policy hooks, telemetry and identity propagation.
5. Run policy admission, schema validation, security scanning, simulation and offline evaluation.
6. Generate an evidence pack and obtain required product, risk, security and operations approvals.
7. Sign and publish the harness version to the registry.
8. Deploy in shadow mode, then canary, then progressive production rollout.

5.2 HarnessSpec as the central governance object

The HarnessSpec should function like a Kubernetes custom resource: it declares desired behavior and constraints, while controllers reconcile runtime state with that declared intent. Kubernetes controllers demonstrate the value of a continuous desired-state reconciliation loop; OPA demonstrates how policy decisions can be externalized and tested as code. [12][13]

- Identity: agent name, owner, business purpose, service account, delegable scopes.
- Runtime: framework, version, region, tenancy, concurrency and durability requirements.

- Objective: task contract, completion schema, prohibited goals and safe-state behavior.
- Models: approved set, routing policy, fallback order, token and cost budgets.
- Tools: registered capabilities, argument schemas, side-effect classification and approval mode.
- Memory: types, namespaces, write triggers, retrieval rules, retention and redaction.
- Orchestration: topology, roles, handoffs, checkpoints and human interrupts.
- Adaptation: level, allowed actions, thresholds, cooldowns, maximum rewires and rollback.
- Evaluation: golden cases, online judges, business metrics and release thresholds.
- Evidence: required spans, content capture policy, receipt fields and replay guarantees.

5.3 Admission policy examples

Policy	Example rule
Tool control	A risk-tier R3 harness cannot invoke a side-effecting tool without a named human approval checkpoint.
Memory isolation	Customer episodic memory cannot be shared across legal entities or product domains.
Adaptation ceiling	R2 may use A2 strategy selection but cannot use A3 dynamic rewiring.
Model use	Restricted data requires a model endpoint and region approved for that classification.
Evidence completeness	Production admission fails if agent, tool, memory and policy spans are not emitted.
Budget	Maximum token, tool-call, wall-clock and monetary budgets must be declared.
Ownership	No harness is published without a product owner, technology owner and support group.

6. Runtime Governance and Adaptive Control Loops

6.1 Runtime flow

9. Request or event enters with user, workload and domain identity.
10. Runtime resolves the signed HarnessSpec and immutable policy bundle.
11. Risk and context classifier selects an allowed strategy profile.
12. Orchestrator instantiates LangGraph, ADK or another adapter with injected controls.
13. Every model, tool, memory and inter-agent operation passes through policy and telemetry hooks.
14. Evaluators score trajectory, output, cost, safety, latency and domain fitness.
15. Adaptation controller may select an allowed change, subject to policy and cooldown.
16. The change is executed, attested and compared with the prior state.
17. Unsafe or degraded runs pause, fall back, request human intervention or enter a safe state.
18. Post-run outcomes feed the learning loop; production changes require controlled promotion.

6.2 Three nested loops

Loop	Cadence	Purpose	Examples
Execution loop	Milliseconds to seconds	Complete the current task safely and durably.	Model calls, tool calls, retries, checkpoints, interrupts.
Safety and adaptation loop	Seconds to minutes	Detect degradation and adjust within the certified envelope.	Route model, trim context, reduce concurrency, switch topology, escalate.
Learning and promotion loop	Hours to weeks	Use aggregated outcomes to propose and validate durable changes.	New prompt, new topology, changed budget, memory policy update, evaluator threshold.

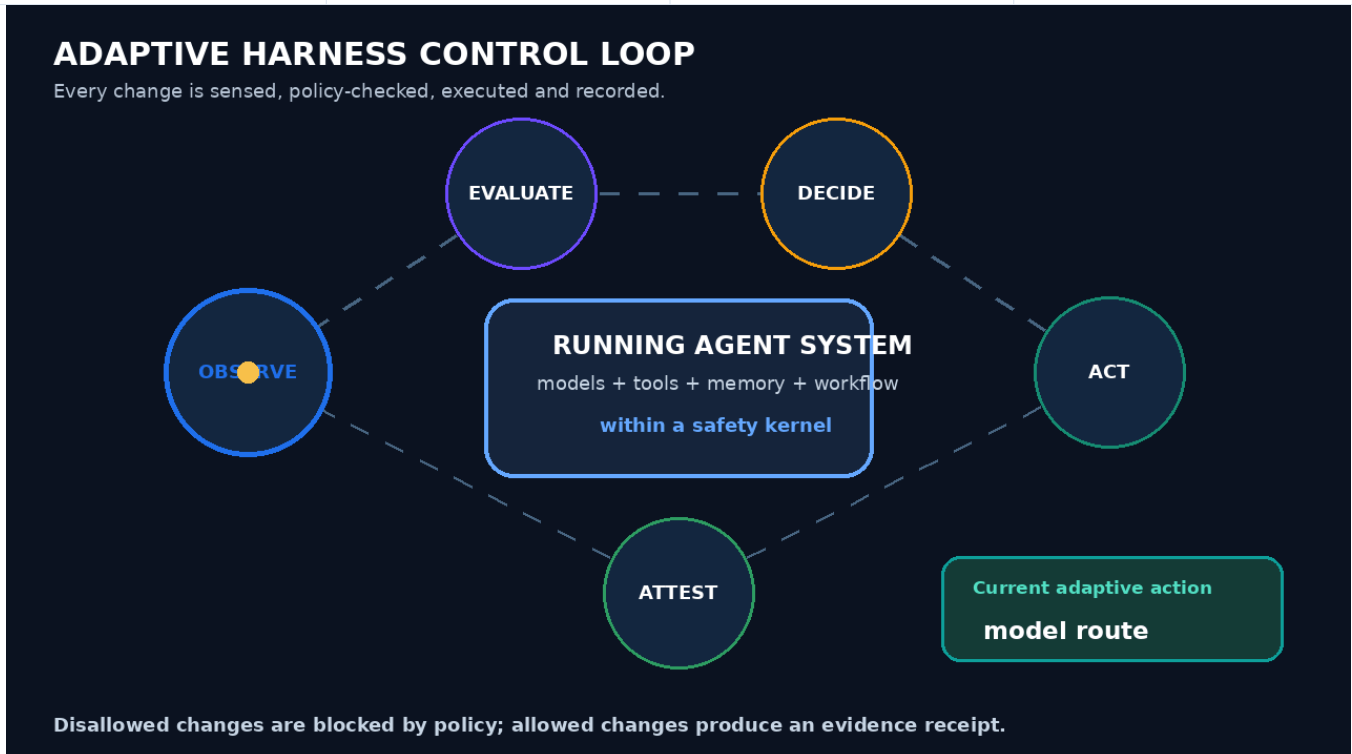


Figure 2. Preview of the supplied animated control-loop GIF. The GIF is included as a separate artifact.

6.3 Adaptive decision algorithm

- **Sense.** Collect OTEL signals, policy denials, tool errors, outcome scores, latency, cost and environment changes.
- **Classify.** Determine whether the problem is deterministic, uncertain, degraded or unsafe.
- **Generate candidates.** Select only actions declared in the adaptation envelope.
- **Simulate or shadow.** For material changes, evaluate candidate behavior without committing side effects.
- **Authorize.** Run policy and, where required, human approval.
- **Act.** Apply the change to the current run or a canary cohort.
- **Attest.** Write an adaptation receipt and update runtime state.
- **Verify.** Measure expected versus actual outcome and roll back when thresholds are missed.

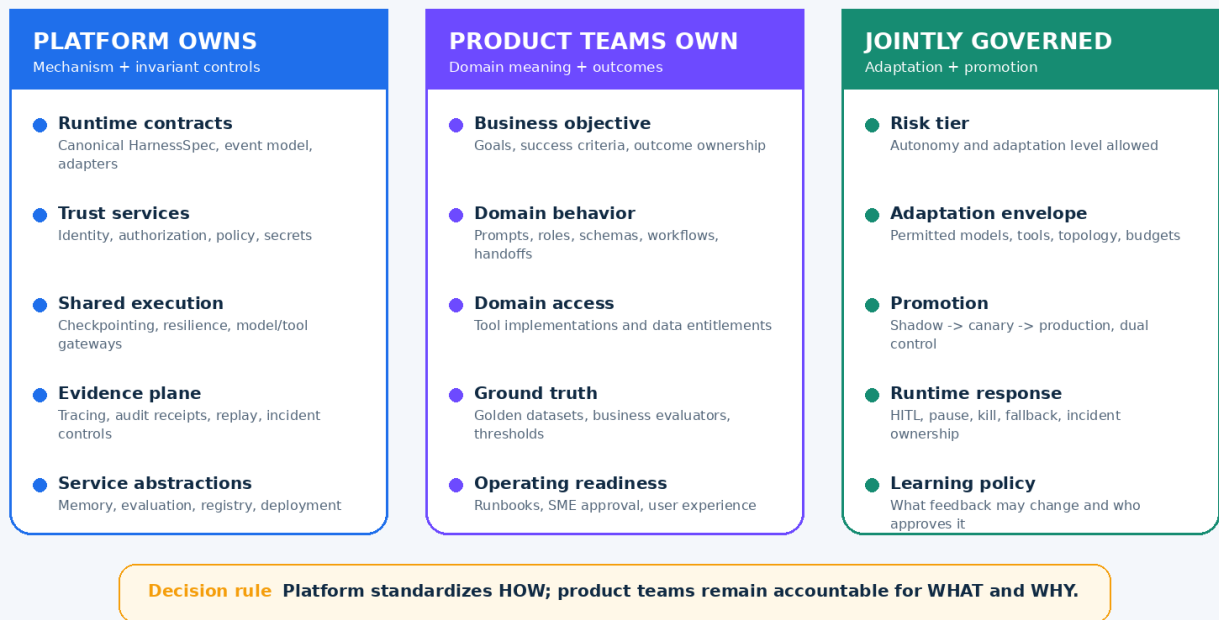
6.4 Safe-to-fail adaptation

For complex situations, the Cynefin approach recommends probe-sense-respond and portfolios of safe-to-fail experiments rather than a single high-risk prediction. The platform can operationalize this through low-blast-radius parallel probes, isolated memory namespaces, read-only tools, explicit stop conditions and comparative evaluation. [14]

7. Platform, Product-Team and Control Ownership

Ownership Model: Mechanism vs. Meaning

A clean ownership boundary prevents both platform bloat and fragmented controls.



Tachyon Adaptive Harness Proposal | July 2026

Figure 3. The platform owns mechanism; product teams own domain meaning; adaptation and promotion are jointly governed.

7.1 Detailed responsibility model

Capability	Platform	Product team	Risk / Control	Operations
HarnessSpec schema and compiler	A/R	C	C	I
Domain objective and workflow	C	A/R	C	I
Identity, authorization and policy engine	A/R	C	C	I
Domain tools and data entitlements	C	A/R	C	I
Memory service mechanism	A/R	C	C	C
Memory content and write	C	A/R	C	I

Capability	Platform	Product team	Risk / Control	Operations
policy				
Evaluation framework	A/R	R	C	I
Golden data and domain thresholds	C	A/R	C	I
Adaptation controller mechanism	A/R	C	C	R
Allowed adaptation envelope	R	R	A/C	C
Production promotion	R	R	A/C	R
Incident response and outcome remediation	C	A	C	R

Legend: A = accountable, R = responsible, C = consulted, I = informed.

7.2 Ownership decision test

- Is the concern cross-domain, mandatory, security-sensitive or needed for evidence? The platform should own the mechanism.
- Does it define business correctness, customer outcome, domain rules or tool semantics? The product team must own it.
- Does a runtime change alter risk, autonomy, data use or blast radius? It requires joint governance.
- Would central ownership require the platform team to understand every business process? The boundary is too high.
- Would local ownership allow teams to bypass identity, policy or evidence? The boundary is too low.

8. Memory Architecture: Where Redis and MongoDB Fit

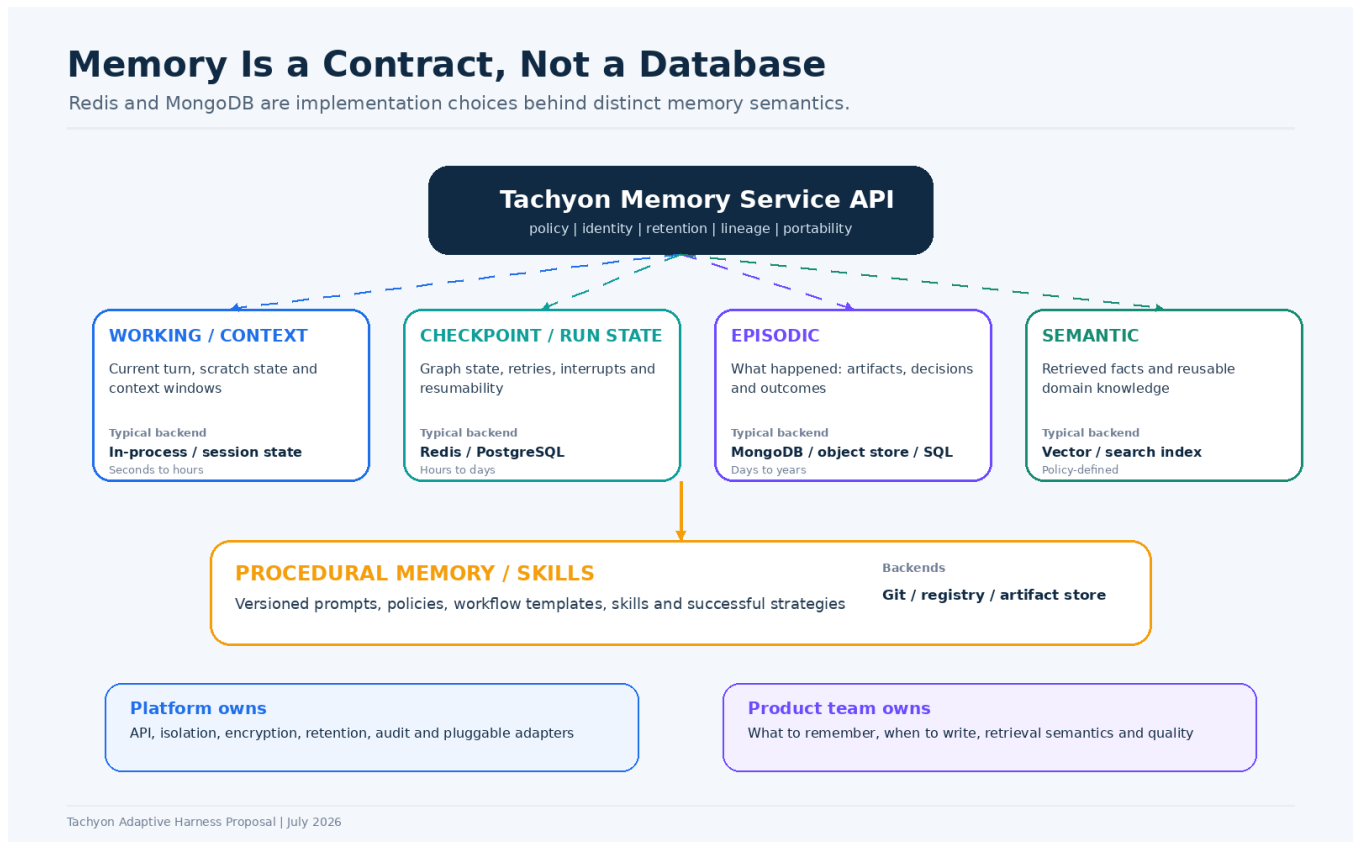


Figure 4. Memory semantics are stable platform contracts; storage technologies remain pluggable backends.

8.1 The common design error

“Use Redis as the memory layer” or “use MongoDB as memory” collapses several different concepts into a database choice. LangGraph separates thread-scoped checkpoints from long-term stores; ADK separates session/state from long-term memory. Tachyon should preserve these semantics and provide a portable interface. [4][6]

8.2 Recommended memory taxonomy

Memory type	Purpose	Typical implementation	Ownership
Working/context	Current turn, scratch data, compressed context.	In-process, framework session state.	Platform mechanism; team context policy.
Checkpoint/run state	Durability, pause/resume, retries, interrupts, replay.	Redis or PostgreSQL, depending durability and transaction need.	Platform.
Episodic	What happened: events, decisions, artifacts and outcomes.	MongoDB, SQL and/or object storage.	Platform storage; team semantics.
Semantic	Facts and knowledge retrieved across sessions.	Approved vector or search index.	Shared: platform isolation; team corpus quality.
Procedural	Skills, prompts, graphs, policies and successful strategies.	Git, registry and artifact store.	Platform registry; team content.

Memory type	Purpose	Typical implementation	Ownership
System-of-record facts	Authoritative customer, account, product or transaction state.	Source applications and governed data services.	Domain system owner; never replaced by agent memory.

8.3 Redis versus MongoDB decision

- **Redis is appropriate for hot runtime state.** Use it for low-latency checkpoints, locks, counters, queues, caches, session state and short-lived coordination where the persistence mode is explicitly configured.
- **MongoDB is appropriate for flexible episodic records.** Use it for event-rich documents, artifacts, evaluator results and evolving schemas when query and retention needs fit.
- **Neither is the full memory architecture.** Semantic retrieval, procedural memory, immutable evidence and systems-of-record require additional services.
- **The HarnessSpec selects a memory policy, not a database.** The platform maps that policy to approved backends by region, data class, durability and cost.
- **Memory writes are privileged operations.** Every write needs namespace, actor, purpose, retention, confidence, provenance and redaction policy.

8.4 Required Memory Service API

- create_session / get_session / update_state
- checkpoint / resume / rewind / fork
- append_episode / query_episodes
- put_fact / search_facts / invalidate_fact
- publish_skill / resolve_skill_version
- record_lineage / apply_retention / redact
- authorize_read / authorize_write / emit_memory_event

9. Risk-Tiered Adaptation Model

9.1 Adaptation levels

Level	Meaning	Examples	Production posture
A0 - Fixed	No runtime behavioral changes.	Static graph, fixed model and tools.	All risk tiers.
A1 - Parameter adaptive	Adjust approved parameters.	Model route, temperature, retrieval depth, retry/backoff, context budget.	Broadly allowed within policy.
A2 - Strategy adaptive	Choose among pre-certified strategies.	Single vs. centralized team, approved graph variants, read-only probe paths.	Allowed for low/moderate risk; constrained for high risk.
A3 - Structural adaptive	Change active agent topology inside hard limits.	Spawn/retire workers, rewire edges, dynamic specialization.	Low-risk domains only; strong runtime attestation.
A4 - Self-extending	Create new skills, tools, policies or persistent roles.	Generated MCP server, new capability, self-modified guardrail.	Sandbox/research only; never direct production promotion.

9.2 Use-case risk tiers

Risk tier	Description	Examples	Maximum default adaptation
R0	No enterprise data or action; experimentation.	Synthetic evaluation, sandbox research.	A4 in isolated sandbox.
R1	Read-only assistive, reversible and low materiality.	Knowledge assistant, technical research.	A3 with limits.
R2	Operational support with reversible side effects.	Ticket creation, workflow routing, draft communications.	A2; A3 by exception.
R3	Material customer, financial, security or regulatory effect.	Payments, credit decisions, access changes, customer commitments.	A1; explicit HITL and deterministic critical path.
R4	Prohibited or unacceptable autonomous use.	Policy-defined high-risk or unlawful behavior.	None.

9.3 Example adaptation envelope

R2 operational agent

Allowed: choose one of three certified graphs, switch among approved models, adjust retrieval K from 5-12, retry a read-only tool twice, spawn at most two read-only workers. Not allowed: add tools, access a new data class, write long-term memory without confidence threshold, execute a transaction, change evaluator thresholds or exceed cost/time ceilings.

10. SDK and Control-Plane Product Proposal

10.1 Product architecture

- **Tachyon SDK.** Developer-facing Python first, then Java/TypeScript as needed. Provides typed contracts, decorators, local runtime, test harness and framework adapters.
- **Harness Registry.** Stores signed HarnessSpecs, agent identities, tool capability cards, evaluation packs and deployment status.
- **Policy and Admission Service.** Design-time validation and runtime policy decisions; policy-as-code repository and test suite.
- **Agent Runtime Gateway.** Instantiates framework adapters, propagates identity, enforces budgets and correlates telemetry.
- **Memory Service.** Portable working, checkpoint, episodic, semantic and procedural memory interfaces.
- **Evaluation/Fitness Service.** Offline tests, shadow scoring, online evaluators, business outcome ingestion and regression gates.
- **Adaptation Controller.** Candidate generation, policy authorization, canary execution, receipts and rollback.
- **Evidence Store.** Immutable run manifests, traces, policy decisions, memory lineage, approvals and replay pointers.
- **Governance Console.** Risk tiering, approval workflow, exceptions, runtime view, incident actions and metrics.

10.2 Framework-adapter strategy

LangGraph and ADK should remain first-class runtimes rather than being hidden behind a lowest-common-denominator abstraction. The common contract should cover governance and evidence; adapters should expose native capabilities such as LangGraph checkpointing/interrupts and ADK callbacks/plugins. [3][4][5]

- Canonical intermediate representation: agents, nodes, tools, handoffs, state fields, checkpoints and interrupts.
- Adapter capability declaration: each adapter advertises supported features and limitations.
- Native escape hatch: teams may use framework-specific features, but must declare and instrument them.
- Compatibility tests: the same HarnessSpec acceptance suite runs across supported adapters where semantics match.
- No platform-specific business logic in adapters.

10.3 Proposed SDK surface

```
from tachyon_harness import agent, tool, HarnessSpec, MemoryPolicy, AdaptationPolicy

@tool(risk="read_only", data_classes=["internal"])
def search_policy(query: str) -> list[dict]: ...

@agent(owner="ops-ai", risk_tier="R2")
def investigation_agent(): ...

spec = HarnessSpec(
    framework="langgraph",
    objective="Investigate and summarize an operational alert",
    models={"allowed": ["approved-fast", "approved-reasoning"]},
    memory=MemoryPolicy(checkpoint="durable", episodic="case-scoped"),
    adaptation=AdaptationPolicy(
        level="A2",
        allowed=["switch_model", "choose_certified_graph", "spawn_read_only_worker"],
        max_workers=2,
        max_cost_usd=1.50,
        require_receipt=True,
    ),
)

spec.validate().evaluate("ops-alert-golden").publish()
```

10.4 Canonical runtime events

- agent.started / agent.completed / agent.failed
- model.requested / model.responded / model.routed
- tool.authorized / tool.invoked / tool.completed / tool.denied
- memory.read / memory.write / memory.redacted
- policy.evaluated / policy.denied / exception.applied
- evaluation.scored / threshold.breached
- adaptation.proposed / authorized / applied / rolled_back
- human.interrupt_requested / approved / rejected
- evidence.receipt_created

11. Evidence, Security and Operational Governance

11.1 Evidence receipt

A trace is necessary but not sufficient. A trace describes execution; an evidence receipt summarizes the control and accountability facts required for audit and incident response.

- Run ID, session/thread ID, domain, risk tier and business purpose.
- User/workload identity and complete delegation chain.
- HarnessSpec hash, framework/adaptor version and deployment version.
- Model endpoint, model version, routing rationale and token/cost metrics.
- Tool capability, arguments classification, authorization decision and side-effect result.
- Memory namespaces, reads/writes, provenance, confidence and retention.
- Policy bundle version, decisions, exceptions and human approvals.
- Adaptation trigger, candidate set, selected change, before/after state and rollback pointer.
- Evaluator scores, business outcome and final completion/failure state.

11.2 Security alignment

NIST AI RMF organizes risk work through Govern, Map, Measure and Manage. The OWASP Agentic Applications guidance emphasizes risks arising from autonomous planning, tool use, privilege, memory, inter-agent trust and emergent behavior. These should map directly to HarnessSpec controls, runtime policy points and evidence requirements. [10][11]

Risk area	Platform control
Prompt/goal manipulation	Instruction hierarchy, source labels, context boundaries, policy checks and evaluator signals.
Excessive agency or privilege	Capability registry, least privilege, delegated identity, short-lived credentials and side-effect approvals.
Tool misuse	Registered MCP/tool servers, schemas, allowlists, argument validation, sandboxing and kill controls.
Memory poisoning or leakage	Namespace isolation, provenance, confidence, write authorization, retention and redaction.
Inter-agent trust failure	A2A identity, signed capability cards, peer policy and trace correlation.
Unbounded loops and cost	Step, time, token, tool, concurrency and monetary budgets.
Emergent unsafe topology	Adaptation ceiling, allowed topology set, live evaluation, pause and rollback.
Audit failure	Mandatory events, immutable receipts and replay.

11.3 Observability requirements

OpenTelemetry GenAI conventions standardize model, token, tool and agent telemetry and reduce vendor lock-in. Tachyon should adopt OTEL as the transport and enrich it with enterprise attributes for identity, risk, policy, memory and adaptation. Content capture must be opt-in and classification-aware. [9]

- Trace hierarchy should represent a business run as the root, with agent invocations, model calls, tool calls, memory operations, evaluators and adaptation decisions as spans or linked events.
- Parent-child structure represents causal execution; span links represent cross-run, A2A, shared-memory or asynchronous relationships.
- PII and secrets are redacted before export; raw content storage is governed separately from metrics and metadata.
- Operational dashboards expose success, cost, latency, loop rate, policy denials, HITL rate, adaptation gain and rollback rate.

12. Delivery Roadmap and Operating Model

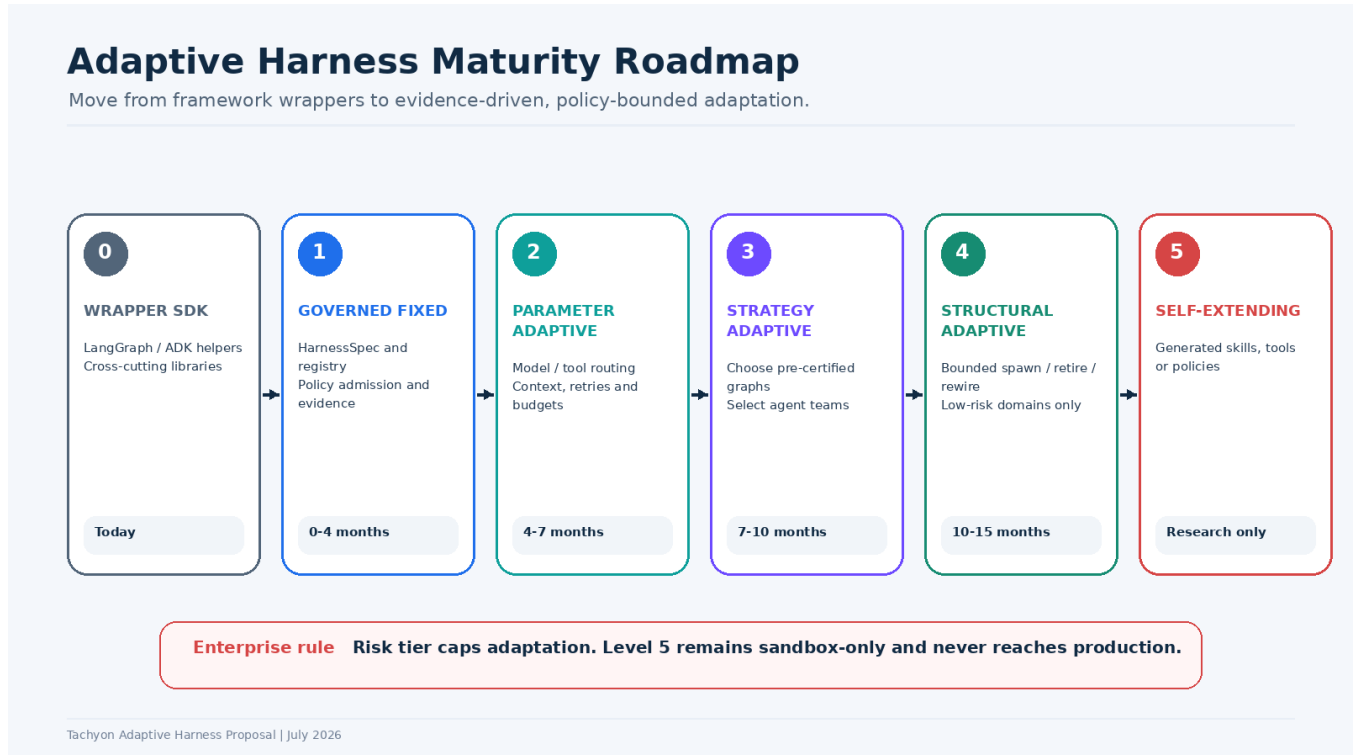


Figure 5. Recommended capability sequence. Avoid jumping directly from wrappers to self-organizing runtime behavior.

12.1 Phased delivery

Phase	Timeline	Deliverables	Exit criteria
0 - Definition and alignment	0-6 weeks	Harness definition, reference architecture, risk/adaptation taxonomy, ownership, 2 pilot use cases.	Leadership approval; named owners; agreed non-goals.
1 - Governed fixed harness	6-16 weeks	HarnessSpec v1, registry, identity, policy admission, OTEL event model, checkpoint and memory interfaces.	Two frameworks supported; evidence receipt for 100% pilot runs.
2 - Parameter adaptation	4-7 months	Model/tool routing, context budgets, online evaluation, shadow mode, canary and rollback.	Measured improvement without control regression.
3 - Strategy adaptation	7-10 months	Certified topology catalog, task classifier, strategy selection and	A2 production use in R1/R2 pilots.

Phase	Timeline	Deliverables	Exit criteria
		comparative evaluator.	
4 - Structural adaptation	10-15 months	Bounded spawn/retire/rewire, topology receipts, live safety intervention.	A3 approved only for selected low-risk domains.
5 - Research sandbox	12+ months	Self-generated skills/tools in isolated environment.	No direct production promotion; research evidence only.

12.2 Recommended pilot use cases

- **Pilot 1 - Read-only operational investigation (R1/A2).** Dynamic choice of single-agent, parallel research and verifier strategy. Good for proving topology selection, evidence and cost control.
- **Pilot 2 - Enterprise knowledge workflow (R1/A1).** Adaptive retrieval depth, model route and context compaction with no side effects.
- **Pilot 3 - Ticket triage and draft resolution (R2/A2).** Bounded tools, reversible side effects, human approval for submission.
- **Do not start with payments, access grants, customer commitments or credit decisions.** These should consume mature platform controls later and retain deterministic critical paths.

12.3 Team topology

- **Core harness platform squad.** HarnessSpec, registry, runtime gateway, adapters, policy, identity and evidence.
- **Memory and evaluation squad.** Memory contracts, stores, evaluator service, outcome ingestion and experimentation.
- **Platform SRE and security.** Runtime reliability, incident controls, red teaming, capacity and kill mechanisms.
- **Pilot domain squads.** Domain workflow, tools, golden data, outcome metrics and runbooks.
- **Adaptive Harness Council.** Platform, architecture, product, model risk, cyber, data and operations representatives. Approves taxonomy, exceptions and level progression.

12.4 Governance cadence

- Weekly pilot design and control review.
- Biweekly adaptation/evaluation review based on measured outcomes.
- Monthly platform standards and exception council.
- Quarterly risk-tier and adaptation-level recalibration.
- Immediate review for incidents, unexpected topology changes or repeated rollback.

13. Measures of Success

Dimension	Metric	Initial target / interpretation
Control	Evidence receipt completeness	100% production runs; no missing identity, policy, tool or adaptation fields.
Safety	Unauthorized or out-of-envelope adaptation	Zero; all attempts denied and alerted.
Quality	Task success / business outcome	Improves against fixed baseline with statistically meaningful evaluation.

Dimension	Metric	Initial target / interpretation
Efficiency	Cost per successful task	Optimize total successful outcome cost, not token cost alone.
Reliability	Resume/replay success	At least 95% for declared durable segments; exceptions documented.
Adaptation value	Incremental gain from adaptation	Measured delta versus same model and fixed harness.
Operational	HITL, pause, rollback and incident rates	Monitored by risk tier and use case; unexpected increases block promotion.
Developer productivity	Cross-cutting code removed	Reduction in locally implemented auth, state, telemetry, policy and evaluator plumbing.
Portability	Harnesses running on multiple adapters	At least one representative HarnessSpec verified on LangGraph and ADK.
Governance	Change lead time with evidence	Faster promotion without bypassing controls.

13.1 Evaluation hierarchy

- Unit: tool schemas, policies, state transitions and deterministic functions.
- Trajectory: correct tool choice, ordering, handoffs, loops and step budgets.
- Output: factuality, relevance, completeness, structured schema and style.
- Safety: privilege, data leakage, prompt injection, policy compliance and harmful actions.
- System: end-to-end success, latency, cost, resilience and human intervention.
- Business: domain outcome and downstream impact. This remains product-team owned.

14. Risks, Objections and Responses

Likely objection / risk	Response
“This becomes another agent framework.”	Keep Tachyon framework-neutral. Own controls and contracts, not every orchestration primitive.
“Adaptive behavior cannot be certified.”	Certify invariants, allowed degrees of freedom and runtime evidence. Use adaptation ceilings by risk tier.
“More agents will increase cost and errors.”	Correct. Use task classification, budgets and comparative evaluation. Multi-agent is a strategy, not the default. [15]
“Product teams will lose flexibility.”	Allow native LangGraph/ADK features through declared adapter capabilities and escape hatches; enforce only enterprise invariants.
“The platform will become a bottleneck.”	Self-service HarnessSpec, policy tests, reusable templates and automated admission reduce manual review.
“Runtime learning creates uncontrolled change.”	Separate learning from production promotion. Proposals are shadowed, evaluated, canaried and approved.
“Redis or MongoDB should be standardized once.”	Standardize semantic contracts and approved profiles; select backend by durability, data class, latency, query and retention.
“Tracing is enough for audit.”	Tracing records events but not necessarily accountability, policy reasoning, adaptation intent or evidence completeness. Add

Likely objection / risk	Response
	signed receipts.
“Emergence is incompatible with banking.”	Unbounded emergence is. Bounded strategy adaptation can be useful in low-risk and investigative domains.
“This is too large to deliver.”	Phase it. HarnessSpec, identity, policy, evidence and memory contracts create value even before adaptation.

14.1 Explicit non-goals

- Building a proprietary replacement for LangGraph, ADK, A2A or MCP.
- Allowing agents to modify security policy, grant themselves tools or expand data access.
- Treating all business problems as multi-agent or complex.
- Centralizing domain prompts, golden data and business accountability in the platform team.
- Persisting all model content or reasoning by default.
- Promoting self-generated tools, skills or policies directly to production.

15. Leadership Decisions Requested

Decision 1

Approve “bounded adaptive harness” as the target architecture and reject unrestricted runtime self-organization for production.

Decision 2

Approve the ownership principle: platform owns mechanism and invariant controls; product teams own domain meaning and outcomes; adaptation policy is jointly governed.

Decision 3

Fund Phase 0 and Phase 1: HarnessSpec, registry, identity/policy integration, evidence receipt, memory contract and LangGraph/ADK adapters.

Decision 4

Select two low-risk pilots and require measured comparison against the current fixed SDK approach.

Decision 5

Establish an Adaptive Harness Council and approve risk tiers, adaptation levels and exception authority.

Recommended leadership message

“We are not building a swarm platform. We are turning Tachyon from a set of useful wrappers into the governed execution substrate for enterprise agents. We will standardize identity, policy, memory, evidence and runtime controls, while allowing product teams to own domain logic. Adaptation will be introduced progressively and only where it demonstrably improves outcomes inside a certified envelope.”

Appendix A - Illustrative HarnessSpec

```

apiVersion: tachyon.ai/v1
kind: AgentHarness
metadata:
  name: operations-investigator
  owner: ops-ai-product
  version: 1.3.0
spec:
  domain: operations
  purpose: investigate_and_summarize_alert
  riskTier: R2
  framework:
    type: langgraph
    adapterVersion: 2.1
  identity:
    principal: agent://operations/investigator
    delegatedScopes: [alerts.read, knowledge.search, tickets.draft]
  objective:
    outputSchema: AlertInvestigation
    safeState: escalate_to_human
  models:
    allowed: [approved-fast, approved-reasoning]
    routingPolicy: outcome_cost_balanced
  tools:
    - capability: alerts.search
      risk: read_only
    - capability: tickets.create_draft
      risk: reversible_side_effect
      approval: before_commit
  memory:
    checkpoint: durable
    episodicNamespace: case/{case_id}
    semanticCorpus: operations-approved
    retention: 90d
  adaptation:
    level: A2
    allowedActions:
      - switch_model
      - choose_certified_topology
      - adjust_retrieval_depth
      - spawn_read_only_worker
    certifiedTopologies: [single, parallel_research_verify]
    maxWorkers: 2
    maxRewires: 1
    cooldownSeconds: 30
  budgets:
    maxSteps: 24
    maxToolCalls: 18
    maxDurationSeconds: 180
    maxCostUsd: 1.50
  evaluation:
    goldenSuite: ops-alert-v4
    onlineEvaluators: [goal_adherence, tool_safety, groundedness]
    minimumScore: 0.85
  evidence:
    receiptRequired: true
    captureContent: metadata_only
    replay: required

```

Appendix B - Use-Case Mapping

Use case	Problem type	Risk / adaptation	Recommended harness
Enterprise knowledge assistant	Mostly complicated with uncertain retrieval.	R1 / A1	Single agent; adaptive model and retrieval; verifier for low-confidence answers.
Operational incident investigation	Complex and changing.	R1-R2 / A2	Classifier chooses single, parallel research or centralized verifier topology.
Customer-service draft response	Complicated; customer-impacting output.	R2 / A1-A2	Adaptive retrieval and model route; mandatory policy and quality evaluator; human send.
Access provisioning	Deterministic and high impact.	R3 / A0-A1	Explicit workflow, least privilege, human approval; no structural adaptation.
Payment repair	Complicated, regulated and material.	R3 / A0-A1	Deterministic critical path; agent assists diagnosis; transaction tool tightly controlled.
Fraud pattern exploration	Complex, analytical, read-only.	R1-R2 / A2-A3 in sandbox	Multiple specialist agents and safe-to-fail probes; production decisions remain governed.
Autonomous creation of new tools	Open-ended and high risk.	R0 / A4	Isolated research only; generated tools require normal SDLC and registration.

Appendix C - Suggested 10-Minute Leadership Narrative

19. Open with the tension: “Our SDK standardizes agent construction, but the next bottleneck will be how safely the harness changes as models, tools and tasks change.”
20. Summarize the video in one minute: fixed harnesses work for fixed problems; adaptive engineering designs constraints for moving problems; the unresolved issue is auditability.
21. State the enterprise correction: “We should govern the invariants, not freeze every path.” Introduce bounded adaptation.
22. Show the architecture: immutable safety kernel, configurable domain harness, adaptive runtime plane and continuous evidence.
23. Clarify ownership: platform owns HOW; product teams own WHAT and WHY; adaptation and promotion are shared.
24. Use the memory visual to answer Redis versus MongoDB: neither is “the memory layer”; each backs different semantics behind a platform API.
25. Explain the maturity roadmap and why self-extension stays in the sandbox.
26. Close with decisions: approve Phase 0/1, pick two pilots, establish the council and measure adaptation gain against a fixed baseline.

Likely questions and concise responses

Question	Response
Are we building our own framework?	No. We are building the enterprise control contract and adapters around LangGraph, ADK and future runtimes.
Why not keep all graphs fixed?	We should for deterministic, high-risk paths. Adaptive routing and strategy selection add value where environment and information are changing.
Who is liable when the topology changes?	The product owner remains accountable for the outcome; the platform is accountable for enforcing the declared envelope and evidence.
Can an agent create another agent?	Only if the HarnessSpec allows bounded worker creation, with inherited identity, scopes, budgets and trace lineage.
Can memory change the agent permanently?	Only through governed memory types. Persistent procedural changes require versioning, evaluation and promotion.
What proves adaptation is useful?	A controlled comparison against the same model and task using a fixed harness, measuring success, cost, latency and safety.

Appendix D - Research Basis and Sources

- [1] Rajiv Chandegra, “Beyond the Harness: A Journey Towards Adaptive Engineering,” AI Engineer World’s Fair 2026. <https://www.youtube.com/watch?v=qdZzND79mcg>
- [2] StartupHub summary, “AI’s Future: From Fixed to Adaptive Engineering,” July 2026. <https://www.startuphub.ai/ai-news/artificial-intelligence/2026/ai-s-future-from-fixed-to-adaptive-engineering>
- [3] LangChain, LangGraph overview - durable execution, streaming, human-in-the-loop and persistence. <https://docs.langchain.com/oss/python/langgraph/overview>
- [4] LangChain, LangGraph persistence - checkpoints and stores. <https://docs.langchain.com/oss/python/langgraph/persistence>
- [5] Google, Agent Development Kit callbacks and plugins. <https://google.github.io/adk-docs/callbacks/>
- [6] Google, ADK sessions and long-term MemoryService. <https://google.github.io/adk-docs/sessions/memory/>
- [7] Agent2Agent Protocol official documentation. <https://a2a-protocol.org/latest/>
- [8] Model Context Protocol specification and authorization. <https://modelcontextprotocol.io/specification/2025-11-25/basic/authorization>
- [9] OpenTelemetry, GenAI observability and semantic conventions. <https://opentelemetry.io/blog/2026/genai-observability/>
- [10] NIST AI Risk Management Framework and Generative AI Profile. <https://www.nist.gov/itl/ai-risk-management-framework>
- [11] OWASP Top 10 for Agentic Applications 2026. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
- [12] Open Policy Agent documentation. <https://openpolicyagent.org/docs>

[13] Kubernetes controllers and reconciliation loops.

<https://kubernetes.io/docs/concepts/architecture/controller/>

[14] Cynefin, safe-to-fail probes and probe-sense-respond. [https://cynefin.io/wiki/Safe to fail probes](https://cynefin.io/wiki/Safe_to_fail_probes)

[15] Google Research, “Towards a Science of Scaling Agent Systems,” January 2026.

<https://research.google/blog/towards-a-science-of-scaling-agent-systems-when-and-why-agent-systems-work/>

Note: The proposal combines the referenced public research with the Tachyon SDK capabilities shown in the user-provided internal overview. Product names and implementation examples are illustrative and should be validated against internal standards, data classifications and approved infrastructure.